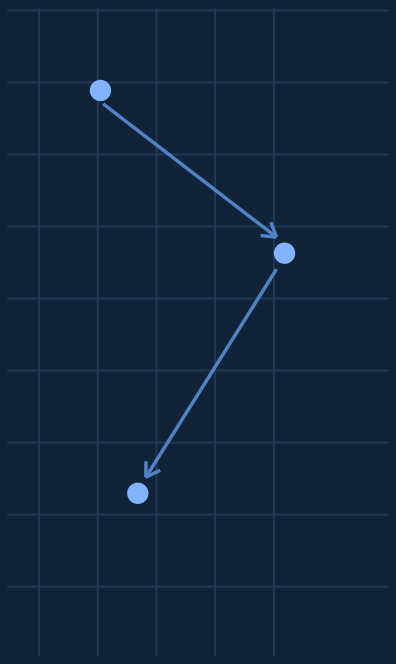
SOFTWARE DEVELOPER

규칙을 코드로,
상태를 경험으로.



장지욱

C++ · C# · Unreal Engine 개발

게임 규칙과 상호작용을 C++로 구현하고,
Unreal Engine의 입력·상태 복제·애니메이션을 연결합니다.
C# 프로젝트에서는 데이터와 이벤트 흐름을 다룹니다.

github.com/phjang0408

SELECTED PROJECTS

01	**도와줘! 왓슨!** Unreal 협동 추리 · 세션 및 API 연동	02-04
02	**DoJokGu** 서버 권한 판정 · 경기 상태 동기화	05-06
03	**YachtDice** 클래스 책임 분리 · 턴 상태 관리	07
04	**Smart_Road** C# 가상 센서 데이터 · 이벤트 연동	08
05	**ShooterX** 입력 · 카메라 · 애니메이션 연동	09

프로젝트의 공개 코드와 팀 최종보고서를 바탕으로 구현 구조와 검증 범위를 정리했습니다. 팀 시스템과 개인 담당이 확인된 부분을 구분했습니다.

자료 확인 2026.09.22

도와줘! 왓슨!

역사 속 사건을 함께 탐색하고 추리하는 협동 게임

C++ | Unreal Engine 5 | Steam | 3인 팀 프로젝트

황옥 사건을 배경으로 증거를 수집하고 AI 조수와 대화하며 결론을 제출합니다. 게임 안의 세션과 상태를 외부 서비스에 연결한 캡스톤 프로젝트입니다.



최종보고서에 수록된 실제 플레이 화면 · Windows 배포본 제공

1–4인
보고서 기준 플레이 구성

10종
수집 가능한 증거물

9개
보고서의 통합시험 항목

PUBLIC C++ IMPLEMENTATION

공개 C++의 중심은 OnlineSessionManager의 방 생성·검색·참가·종료와 GameState의 HTTP/JSON 연동입니다. 맵·UI·퍼즐과 AI 백엔드는 팀 결과물로 소개합니다.

PROJECT & REFERENCES

Project_Capstone / 소스 코드

HelpWatson / Windows 배포본

화면·시험 기록: 팀 최종보고서「도와줘! 왓슨!」(2025.12.10)

게임과 서비스를 연결하기

세션 수명주기 · 방 컨텍스트 · 비동기 응답 처리

UGameInstanceSubsystem | HTTP / JSON | RPC / Replication

플레이어가 접속한 게임 세션과 백엔드의 방 컨텍스트를 Room-Id로 연결합니다. HTTP 응답을 Unreal 상태와 Blueprint 이벤트로 전달해 화면이 사용할 수 있도록 구성했습니다.

SESSION & RESPONSE FLOW



상단: 공개 C++의 연결

우측 하단: 보고서 구성

완료 이벤트를 잇는 세션 관리

기존 세션이 있으면 종료 Delegate에서 생성·참가를 이어갑니다. 참가 완료 후 접속 문자열을 얻어 ClientTravel을 수행하며, 호스트 종료와 참가자 이탈을 구분합니다.

JSON 응답을 게임 상태로 변환

비동기 완료 콜백에서 result의 답변·점수·등급·피드백을 읽습니다. AI 답변은 Multicast와 Blueprint 이벤트로 전달하고, 최종 결과는 복제 프로퍼티에 저장합니다.

IMPLEMENTATION BOUNDARY

공개 코드에서 복제 등록이 확인되는 값은 AI 답변과 최종 점수·등급·피드백입니다. 공유 인벤토리의 전체 동기화는 Blueprint 원본 확인이 필요합니다. 현재 호스팅 코드는 ?listen 경로를 사용합니다.

CODE & DOCUMENTATION

OnlineSessionManager.cpp
세션 인터페이스

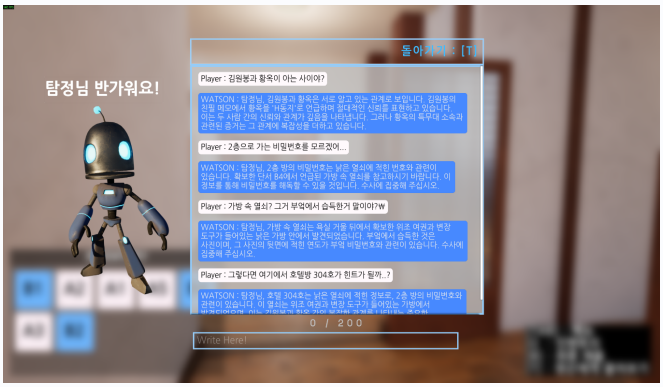
AMyGameStateBase.cpp
게임 상태와 복제 필드

탐색에서 추리 결과까지

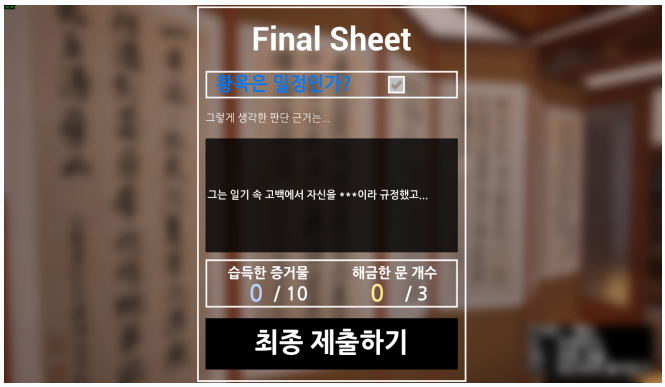
실제 화면과 최종보고서의 통합시험 기록

증거 수집 | AI 대화 | 최종 제출 / 채점

증거를 해석하고 AI 조수와 질문을 주고받은 뒤, 사건에 대한 결론과 근거를 제출합니다. 최종 결과는 증거 점수와 추론 평가를 합산하는 흐름으로 설계했습니다.



AI 조수와 대화



결론과 근거를 작성하는 최종 답안

REPORTED INTEGRATION TESTS / 9 CASES

접속·탐색 / 3개	방 생성 · 방 참가 · 증거 수집/인벤토리
상호작용 / 3개	AI 대화 · 잠금 공간 해금 · 최종 답안 제출
결과·동기화 / 3개	채점 결과 · 멀티플레이 동기화 · 옵션 저장

최종보고서에는 위 9개 항목의 정상 동작 결과가 기록되어 있습니다. 실행 배포본의 압축 파일 목록을 확인했으며, 이 문서 작성 과정에서 게임이나 통합시험을 재실행하지는 않았습니다.

공개 범위: C++ 소스 일부 + Windows 실행 배포본. 재빌드에는 원본 .uproject·Content·Config가 필요하며, AI 대화·채점에는 별도 서비스가 필요합니다.

PROJECT & REFERENCES

Project_Capstone / 소스 코드

HelpWatson / Windows 배포본

화면·시험 기록: 팀 최종보고서「도와줘! 왓슨!」(2025.12.10)

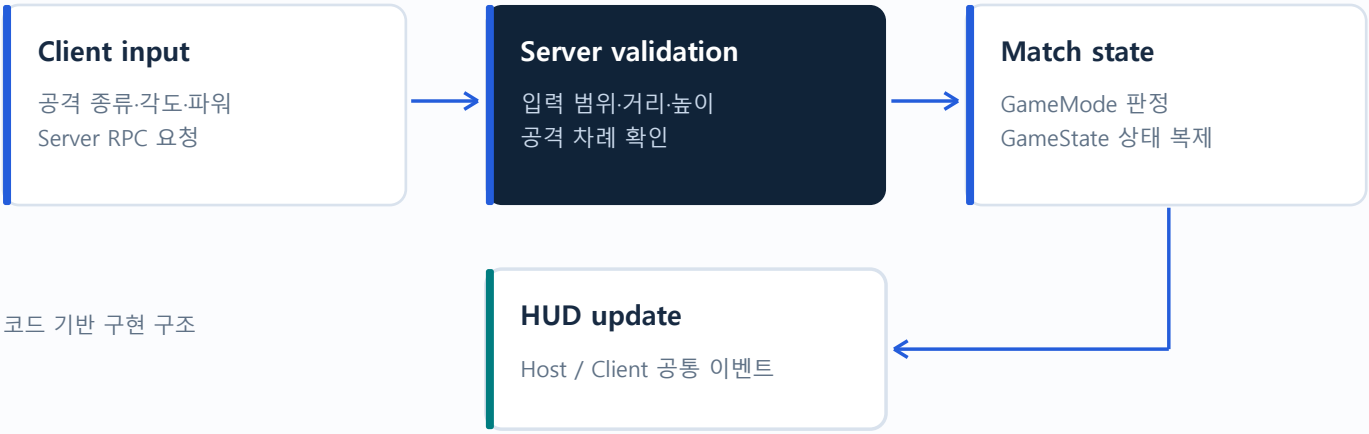
DoJokGu

족구 경기 멀티플레이 프로토타입

C++ | Unreal Engine | RPC / Replication | 개발 중

서브부터 랠리, 득점, 듀스, 재경기까지의 경기 흐름을 구현한 프로젝트입니다. 판정은 서버에서 수행하고, 클라이언트는 복제된 경기 상태를 받아 화면을 갱신하는 구조입니다.

SERVER-AUTHORITATIVE FLOW



코드 기반 구현 구조

판정과 표시의 책임 분리

GameMode가 경기 규칙과 점수 처리를 담당합니다. GameState의 스냅샷을 복제하고, Host의 직접 알림과 Client의 RepNotify가 같은 변경 이벤트로 이어집니다.

중복 입력·중복 득점 방지

공격 입력을 한 슬롯에 저장하고 성공 시 소비합니다. 라운드별 득점 플래그와 랠리 초기화로 같은 라운드의 점수가 중복 반영되는 것을 막습니다.

접속 수명주기

Listen Server 호스트, 주소 기반 참가, 나가기 기능을 SessionSubsystem에 모았습니다. 네트워크·맵 이동 실패는 공통 실패 이벤트로 전달합니다.

현재 범위: 첫 멀티플레이 경기 프로토타입. 로비·캐릭터 선택·EOS 연동·Android 실기기 검증은 작업 문서상 후속 과제입니다.

CODE & DOCUMENTATION

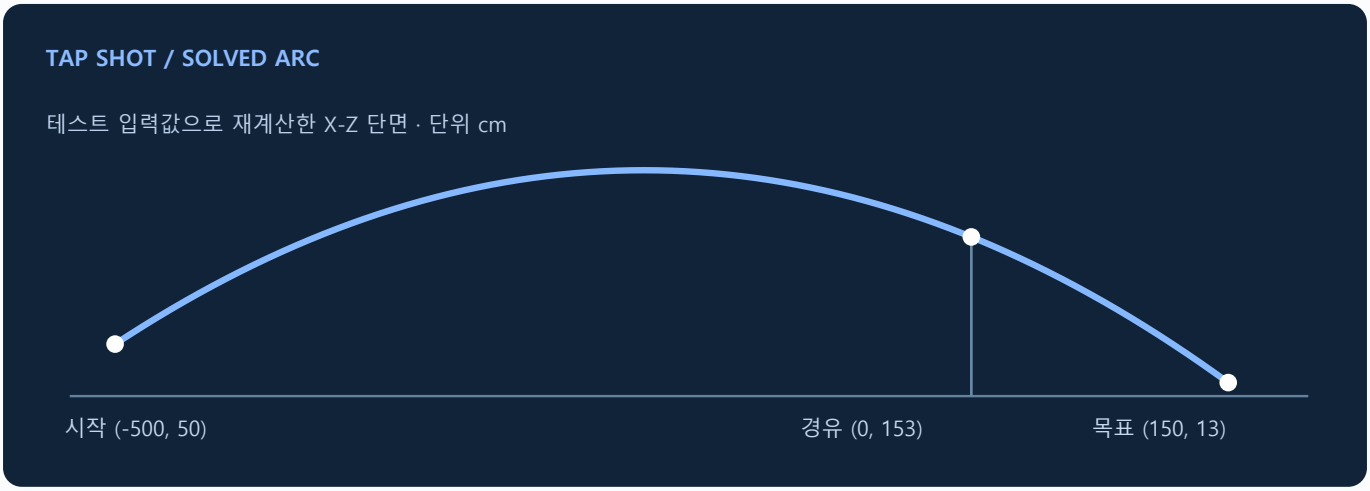
JGGameModeBase.cpp
JGGameStateBase.cpp
JGCharacter.cpp
JGSessionSubsystem.cpp

규칙을 분리하고 검증하기

DoJokGu의 타격 궤적 계산과 경계 조건 테스트

게임 수학 | 순수 규칙 함수 | Unreal Automation Test

경기 판정은 JGMatchRules, 타격 속도 계산은 JGShotCalculator에 분리했습니다. 규칙과 계산을 작은 단위로 나누어 시각 에셋 없이도 경계 조건을 확인할 수 있는 구조입니다.



탭 공격은 네트 위 경유점과 목표 착지점을 지나는 포물선의 초기 속도를 계산합니다. 드래그 공격은 입력 파워를 정규화하고 Curve와 보간을 통해 수평·수직 속도를 계산합니다.

AUTOMATION TESTS IN SOURCE

Rules.Deuce	7:6 경기 지속, 8:6 승리 등 듀스·승리 조건
Rules.CourtZone	코트 라인 위는 인, 경계 밖은 아웃
Rules.GroundContact	첫 착지·반환 실패·랠리 초기화 후 중복 득점
Shot.Calculator	입력 범위·제스처·파워별 속도·경유점 통과

저장소에는 4개 자동화 테스트가 정의되어 있습니다. 작업 문서에는 통과 기록이 있으며, 이 포트폴리오 작성 과정에서 Unreal 테스트를 재실행하지는 않았습니다.

CODE & DOCUMENTATION

JGMatchRules.cpp
JGShotCalculator.cpp
JGMatchRulesTests.cpp
프로토타입 작업 현황

YachtDice

객체의 책임을 나누어 만든 콘솔 주사위 게임

C++ | STL / std::array | 상태 관리 | 학습 프로젝트

C++ 클래스 활용을 연습하기 위해 게임 진행, 주사위, 입력, 점수 계산, 점수 저장, 출력을 나누었습니다. 12턴 동안 주사위 유지와 재굴림, 점수 선택을 반복하는 게임입니다.

RESPONSIBILITY MAP



게임 상태와 턴 상태 분리

메뉴·진행·종료 상태 안에서 시작·굴림·점수 선택·턴 종료 흐름을 처리합니다. 화면 출력과 점수 계산을 각 담당 클래스로 위임합니다.

계산과 저장의 분리

Scorer가 주사위 눈의 빈도 배열을 만들고 카테고리별 예상 점수를 반환합니다. ScoreBoard는 선택된 점수와 사용 여부를 관리합니다.

TURN FLOW

StartTurn → **Rolling** → **SelectScore** → **EndTurn**

최대 3회 굴림 · 점수 미리보기 · 12개 카테고리

설계 포인트: 점수 규칙을 계산하는 코드와 게임을 진행하는 코드를 분리해, 역할별로 읽고 수정할 수 있도록 구성했습니다.

CODE & DOCUMENTATION

프로젝트 목적 및 클래스 구조
GameManager.cpp
Scorer.cpp
ScoreBoard.cpp

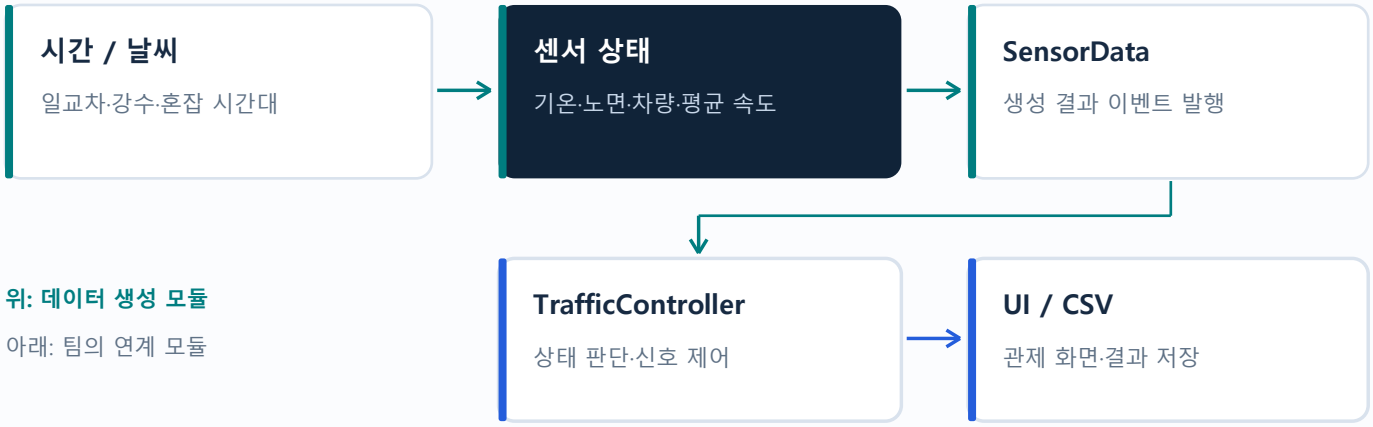
Smart_Road

가상 센서 데이터 기반 교차로 신호 제어 시뮬레이터

C# / .NET 9 | WPF | 이벤트 기반 | 4인 팀 프로젝트

담당: 데이터 생성 모듈. 기상·노면·교통량 사이의 관계를 가상 데이터로 표현하고, 이벤트를 통해 신호 제어 모듈과 연결한 팀 프로젝트입니다.

DATA DEPENDENCY & EVENT FLOW



위: 데이터 생성 모듈

아래: 팀의 연계 모듈

서로 연관된 가상 데이터

코사인 곡선으로 기온의 일주기를 표현하고 기온·강수량으로 노면 상태를 결정합니다. 시간대별 대기 차량 수를 만들고, 대기량에 따라 GPS 평균 속도 범위를 바꿉니다.

시간 진행과 자원 관리

DispatcherTimer로 5분 단위 가상 시간을 진행하고 1·2·4배속을 지원합니다. SensorDataUpdated로 상태를 전달하며, 정지 시 타이머 이벤트를 해제합니다.

288회 업데이트 / 가상 하루

24시간 × 60분 ÷ 5분 · 코드에 정의된 시간 진행 단위

개인 담당: 데이터 생성 및 센서 종속성 구현. 신호 제어·UI·테스트 및 저장은 팀의 별도 담당 모듈입니다.

CODE & DOCUMENTATION

프로젝트 개요 및 역할 분담

SensorData.cs

DataGenerator.cs

Smart_Road.csproj

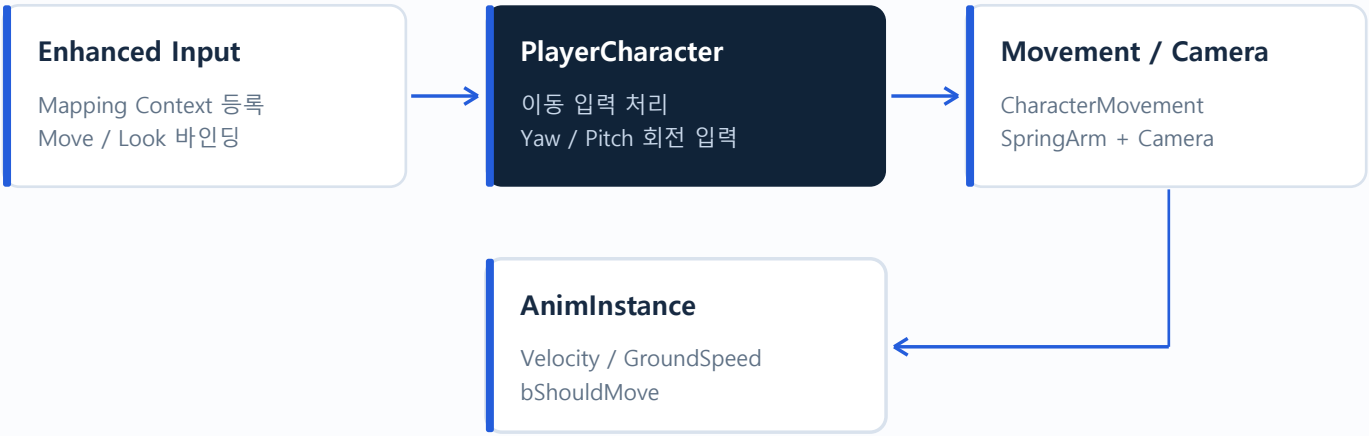
ShooterX

입력·이동·카메라·애니메이션을 연결하는 Unreal 프로젝트

C++ | Enhanced Input | CharacterMovement | 학습 / 개발 중

캐릭터 조작을 입력 매핑에서 이동 컴포넌트, 카메라, 애니메이션 상태까지 연결한 프로젝트입니다. 공개 코드에서 확인되는 조작 기반 기능을 중심으로 소개합니다.

CHARACTER CONTROL PIPELINE



입력 설정과 행동 연결

LocalPlayer의 EnhancedInputSubsystem에 입력 매핑을 등록하고, Move·Look 액션을 각 처리 함수에 바인딩합니다. 캐릭터의 전방·우측 벡터로 이동 방향을 계산합니다.

이동 상태를 애니메이션으로

AnimInstance에서 소유 캐릭터의 이동 컴포넌트를 참조합니다. 평면 속도와 가속도를 계산해 GroundSpeed와 bShouldMove를 갱신합니다.

카메라와 캐릭터의 회전 관계

SpringArm과 Camera를 구성하고, 캐릭터와 카메라의 회전 상속을 설정합니다. SpringArm의 충돌 검사를 활성화해 카메라와 벽 사이의 충돌에 대응하도록 구성합니다.

개발 단계: 3인칭 조작의 기반을 구현하는 학습 프로젝트. 입력·이동·카메라 설정에서 애니메이션 상태 연동까지 이어지는 흐름을 다룹니다.

CODE & DOCUMENTATION

SXPlayerCharacter.cpp
SXAnimInstance.cpp
SXAnimInstance.h
프로젝트 설정